Introducción breve a las expresiones regulares para validación de formularios

ⓘ Propósito de esta nota

En esta nota vas a ver una introducción breve y práctica a las expresiones regulares orientadas a validación de formularios.

El objetivo no es convertir esto en una teoría abstracta de regex, sino entender:

- qué problema resuelven;
- cómo se usan con el atributo `pattern`;
- qué símbolos básicos conviene conocer;
- y en qué casos tienen sentido dentro de un formulario HTML.

---

1. ¿Qué problema intentan resolver?

En muchos formularios no basta con indicar que un campo es obligatorio. A veces también necesitas controlar **el formato** del dato introducido.

Por ejemplo, puede interesarte que:

- un código postal tenga exactamente 5 cifras;
- un nombre de usuario tenga entre 4 y 12 caracteres;
- una contraseña siga unas reglas mínimas;
- un identificador tenga un patrón determinado;
- un teléfono siga un formato básico concreto.

HTML ofrece una primera capa de validación con atributos como:

- `required`
- `minlength`
- `maxlength`
- `min`
- `max`
- `type`

Pero cuando necesitas validar un **patrón de texto**, el atributo más útil suele ser `pattern`.

> **Idea clave**
>
> `pattern` no valida “si el dato tiene sentido en el mundo real”. Valida si el texto escrito **encaja o no** con una forma concreta.

---

2. ¿Qué es una expresión regular?

Una expresión regular, o **regex**, es un patrón que describe cómo debe ser una cadena de texto.

No representa un valor concreto, sino una **regla de forma**.

Ejemplo sencillo

Si escribes este patrón:

```
[0-9]{5}
```

estás indicando:

- debe haber dígitos del 0 al 9;
- y deben aparecer exactamente 5.

Eso encajaría con entradas como:

- `28001`
- `41015`
- `08022`

pero no con:

- `2800`
- `28A01`
- `28001-`

---

3. Cómo se usa con `pattern` en HTML

El atributo `pattern` se coloca normalmente en un `input` de tipo texto o similar.

Ejemplo

```
<label for="postal-code">Código postal</label>
<input
  id="postal-code"
  name="postalCode"
  type="text"
  pattern="[0-9]{5}"
  required
>
```

En este caso, el campo solo será válido si el valor introducido coincide con el patrón.

> **Importante**
>
> `pattern` funciona especialmente bien junto con:
>
> - `required`
> - `minlength`
> - `maxlength`
> - `title`
>
> porque así no solo validas, sino que también orientas mejor a la persona usuaria.

Ejemplo con ayuda adicional

```
<label for="postal-code">Código postal</label>
<input
  id="postal-code"
  name="postalCode"
  type="text"
  pattern="[0-9]{5}"
  title="Introduce un código postal de 5 cifras"
  required
>
```

Aquí `title` puede servir como pista sobre el formato esperado.

---

4. Símbolos básicos que conviene conocer

No necesitas aprender una teoría completa de regex para empezar a validar formularios. Con unos pocos símbolos puedes resolver bastantes casos útiles.

4.1. Rangos entre corchetes

`[0-9]`

Significa: cualquier dígito del 0 al 9.

`[a-z]`

Significa: cualquier letra minúscula de la `a` a la `z`.

`[A-Z]`

Significa: cualquier letra mayúscula de la `A` a la `Z`.

`[a-zA-Z]`

Significa: cualquier letra mayúscula o minúscula.

`[a-zA-Z0-9]`

Significa: cualquier letra o número.

4.2. Cuantificadores

Los cuantificadores indican cuántas veces debe repetirse un patrón.

Símbolo	Significado
`{5}`	exactamente 5 veces
`{2,4}`	entre 2 y 4 veces
`{8,}`	8 o más veces
`?`	0 o 1 vez
`+`	1 o más veces
`*`	0 o más veces

Ejemplos

Patrón	Significado
`[0-9]{5}`	exactamente 5 cifras
`[a-z]{4,12}`	entre 4 y 12 letras minúsculas
`[A-Z]{2}[0-9]{4}`	2 mayúsculas seguidas de 4 cifras

4.3. El punto `.`

El punto representa **cualquier carácter**, salvo ciertas excepciones según contexto.

En validación de formularios conviene usarlo con cuidado, porque puede volver el patrón demasiado permisivo.

> ⚠ **Recomendación**
>
> En validaciones de formularios, suele ser mejor describir con más precisión qué caracteres quieres permitir en vez de usar `.` de forma indiscriminada.

4.4. Anclas `^` y `$`

Estas anclas indican el inicio y el final del texto.

Símbolo	Función
`^`	inicio de la cadena
`$`	final de la cadena

Ejemplo

```
^[0-9]{5}$
```

Esto indica que:

- el texto empieza ahí;
- contiene exactamente 5 dígitos;
- y termina ahí.

> **Matiz útil**
>
> Cuando trabajas con `pattern` en HTML, el navegador ya interpreta que el valor completo debe ajustarse al patrón. Aun así, verás con frecuencia `^` y `$` en ejemplos de regex, así que conviene conocerlos.

4.5. Grupos sencillos con paréntesis

Los paréntesis permiten agrupar partes del patrón.

Ejemplo

```
(HTML|CSS|JS)
```

Esto significa: debe aparecer una de estas tres opciones:

- `HTML`
- `CSS`
- `JS`

El símbolo `|` funciona como una alternativa: “esto o esto otro”.

---

5. Ejemplos orientados a formularios

Aquí tienes algunos casos realistas de uso.

5.1. Código postal de 5 cifras

Patrón

```
[0-9]{5}
```

Campo HTML

```html
<label for="postal-code">Código postal</label>
<input
  id="postal-code"
  name="postalCode"
  type="text"
  pattern="[0-9]{5}"
  title="Introduce un código postal de 5 cifras"
  required
>
```

Qué acepta

- `28001`
- `41015`

Qué rechaza

- `2800`
- `28A01`

5.2. Nombre de usuario sencillo

Requisito

Quieres permitir:

- letras minúsculas;
- números;
- entre 4 y 12 caracteres.

Patrón

```
[a-z0-9]{4,12}
```

Campo HTML

```html
<label for="username">Nombre de usuario</label>
<input
  id="username"
  name="username"
  type="text"
```

```
  pattern="[a-z0-9]{4,12}"
  title="Usa entre 4 y 12 caracteres en minúsculas y números"
  required
>
```

Qué acepta

- `ana7`
- `dev2026`
- `mario123`

Qué rechaza

- `Ana`
- `ab`
- `usuario_largo_demasiado`

> **Observación**
>
> Si quieres permitir guiones o guiones bajos, el patrón tendría que cambiar.

5.3. Teléfono sencillo

Requisito

Quieres aceptar exactamente 9 cifras seguidas.

Patrón

```
[0-9]{9}
```

Campo HTML

```
<label for="phone">Teléfono</label>
<input
  id="phone"
  name="phone"
  type="tel"
  pattern="[0-9]{9}"
  title="Introduce un teléfono de 9 cifras sin espacios"
```

```
  required
>
```

Qué acepta

- `612345678`

Qué rechaza

- `612 345 678`
- `+34612345678`

> ⚠ **Importante**
>
> Este patrón solo sirve para un formato concreto, no para “todos los teléfonos posibles”. Esa diferencia hay que entenderla bien.

5.4. Contraseña básica

Requisito sencillo

Quieres una contraseña con:

- al menos 8 caracteres;
- letras y números.

Patrón simple

```
[a-zA-Z0-9]{8,}
```

Campo HTML

```
<label for="password">Contraseña</label>
<input
  id="password"
  name="password"
  type="password"
  pattern="[a-zA-Z0-9]{8,}"
  title="Usa al menos 8 caracteres con letras y números"
```

```
  required
>
```

Qué acepta

- `abc12345`
- `Test2026`

Qué rechaza

- `abc12`
- `***12345`

> **Matiz importante**
>
> Esto no garantiza una contraseña realmente robusta. Solo impone una regla básica de formato.

5.5. Identificador con formato concreto

Requisito

Dos letras mayúsculas seguidas de cuatro cifras.

Patrón

```
[A-Z]{2}[0-9]{4}
```

Campo HTML

```
<label for="access-code">Código de acceso</label>
<input
  id="access-code"
  name="accessCode"
  type="text"
  pattern="[A-Z]{2}[0-9]{4}"
  title="Usa 2 letras mayúsculas y 4 cifras, por ejemplo AB1234"
  required
>
```

Qué acepta

- `AB1234`
- `ZX0001`

Qué rechaza

- `ab1234`
- `ABC123`
- `A12345`

---

6. Qué conviene enseñar como buenas prácticas

6.1. Usa regex cuando realmente aporte valor

No todos los campos necesitan `pattern`.

Por ejemplo:

- un `type="email"` ya aporta una validación básica útil;
- un `type="number"` con `min` y `max` puede ser suficiente;
- un campo de texto libre no necesita una regex compleja por defecto.

> **Idea clave**
>
> No uses expresiones regulares “porque sí”. Úsalas cuando el formato esperado lo justifique.

6.2. No hagas patrones imposibles de entender

Una regex demasiado críptica puede hacer más daño que bien.

Mal enfoque

- patrón larguísimo;
- casi imposible de leer;
- nadie entiende qué valida exactamente.

Mejor enfoque

- patrón sencillo;
- objetivo claro;
- mensaje de ayuda comprensible.

6.3. Acompaña `pattern` con una ayuda legible

Si un campo exige un formato concreto, debes comunicarlo.

Puedes hacerlo con:

- texto visible junto al campo;
- `aria-describedby`;
- `title`;
- una combinación de varias ayudas.

Ejemplo

```
<label for="code">Código de alumno</label>
<p id="code-help">Debe tener 2 letras mayúsculas y 4 cifras.</p>
<input
  id="code"
  name="studentCode"
  type="text"
  pattern="[A-Z]{2}[0-9]{4}"
  aria-describedby="code-help"
  required
>
```

6.4. No intentes validar el mundo entero con una sola regex

Hay formatos que pueden complicarse muchísimo:

- correos electrónicos;
- teléfonos internacionales;
- contraseñas con reglas muy detalladas;
- nombres personales de todos los idiomas posibles.

En una fase inicial, lo sensato es trabajar con **patrones razonables y pedagógicos**, no con validaciones extremas.

> ⚠ **Error frecuente**
>
> Querer escribir la regex “perfecta” demasiado pronto suele llevar a patrones difíciles de entender, mantener y explicar.

---

7. Qué no conviene complicar todavía

En esta introducción no hace falta entrar en:

- lookaheads;
- lookbehinds;
- grupos de captura avanzados;
- regex complejas para emails;
- validaciones de producción demasiado estrictas.

Aquí lo importante es dominar:

- patrones sencillos;
- lectura básica;
- uso con `pattern`;
- y aplicación útil en formularios.

---

8. Relación entre `pattern`, `title` y accesibilidad

Cuando usas `pattern`, la persona usuaria necesita saber qué formato se espera.

Por eso suele ser buena idea acompañarlo de:

- una explicación visible;
- o un `title`;
- o una ayuda asociada.

Ejemplo completo

```html
<label for="username">Nombre de usuario</label>
<p id="username-help">
  Debe tener entre 4 y 12 caracteres, usando solo minúsculas y números.
</p>
<input
  id="username"
  name="username"
  type="text"
  pattern="[a-z0-9]{4,12}"
  aria-describedby="username-help"
  required
>
```

Criterio correcto

Validar sin explicar el formato esperado genera frustración. La validación debe acompañarse siempre de una orientación clara.

---

9. Tabla resumen de patrones útiles de iniciación

Caso	Patrón	Qué valida
Código postal	`[0-9]{5}`	5 cifras
Teléfono básico	`[0-9]{9}`	9 cifras seguidas
Usuario simple	`[a-z0-9]{4,12}`	4 a 12 caracteres con minúsculas y números
Código de acceso	`[A-Z]{2}[0-9]{4}`	2 mayúsculas y 4 cifras
Contraseña simple	`[a-zA-Z0-9]{8,}`	8 o más letras o números

---

10. Análisis técnico avanzado

10.1. `pattern` valida forma, no significado

Un patrón puede aceptar algo con el formato correcto pero sin sentido real.

Por ejemplo:

- `00000` puede cumplir el patrón de código postal;
- `AAAAAAAA` puede cumplir un patrón de contraseña simple;
- `999999999` puede cumplir un patrón de teléfono.

Eso significa que la regex no sustituye a otras validaciones más profundas.

10.2. HTML y regex ofrecen una primera capa útil

La combinación de:

- `type`
- `required`
- `minlength`
- `maxlength`
- `pattern`

permite construir formularios bastante sólidos para muchos casos sencillos.

10.3. La complejidad debe ser proporcional al contexto

En una práctica formativa o en una interfaz simple, una regex sencilla suele ser mejor que una regex perfecta pero ilegible.

10.4. Las regex también preparan para JavaScript

Aunque aquí las uses con `pattern`, estas nociones te servirán más adelante cuando trabajes validación con JavaScript.

---

11. Conclusión

Las expresiones regulares son una herramienta útil para validar formatos de texto en formularios, pero deben usarse con criterio.

En esta introducción has visto que:

- sirven para describir patrones;
- se integran con `pattern` en HTML;
- pueden ayudarte a controlar formatos comunes;
- y conviene empezar por expresiones sencillas, comprensibles y bien explicadas.

La idea no es memorizar símbolos sin contexto, sino entender cómo pueden ayudarte a construir formularios más claros y más robustos.

12. Resumen de ideas clave

Qué debes retener de esta nota

- Una expresión regular describe un patrón de texto.
- En formularios HTML suele usarse con el atributo `pattern`.
- `pattern` sirve para controlar el formato esperado.
- Los corchetes, cuantificadores y rangos son suficientes para muchos casos básicos.
- No todos los campos necesitan regex.
- Una regex debe ser comprensible y proporcionada al problema.
- Si exiges un formato, debes explicarlo con claridad.
- `pattern` valida forma, no significado real.

13. Referencias documentales generales

1. MDN Web Docs. *HTML attribute: pattern*.
2. MDN Web Docs. *Client-side form validation*.
3. MDN Web Docs. *The input element*.
4. MDN Web Docs. *Regular expressions*.

14. Mini glosario

Término	Definición breve
Expresión regular	Patrón que describe cómo debe ser un texto
`pattern`	Atributo HTML para validar un formato
Cuantificador	Parte de la regex que indica cuántas veces se repite algo
Rango	Conjunto de caracteres permitidos, como `[0-9]`
Ancla	Símbolo que marca inicio o fin del texto
Validación	Comprobación de que un dato cumple una regla

15. Ejemplo integrador

```html
<form action="/register" method="post">
  <label for="student-code">Código de alumno</label>
  <p id="student-code-help">
    Debe tener 2 letras mayúsculas y 4 cifras, por ejemplo AB1234.
  </p>
  <input
    id="student-code"
    name="studentCode"
    type="text"
    pattern="[A-Z]{2}[0-9]{4}"
    aria-describedby="student-code-help"
    required
  >

  <label for="postal-code">Código postal</label>
  <input
    id="postal-code"
    name="postalCode"
    type="text"
    pattern="[0-9]{5}"
    title="Introduce un código postal de 5 cifras"
    required
  >

  <button type="submit">Enviar</button>
</form>
```